

TFM E-Commerce Sales Forecast

Análisis Predictivo y Segmentación de Clientes

Máster en Inteligencia Artificial y Big Data — MP6

Entrega Final - Grupo K1

Jesús Alonso Cendrero

Aleix Bermúdez Arnau

Lucas Chacón Langa

Diego Darachaga Leon

1. Introducción y Contexto del Proyecto.....	4
1.1. Objetivos.....	4
1.2. Repositorio de Github.....	4
1.3. Dataset utilizado.....	4
1.4. Stack tecnológico.....	5
2. Preprocesado de Datos.....	6
2.1. Carga inicial y análisis exploratorio (EDA).....	6
2.1.1. Carga del dataset.....	6
2.1.2. Análisis exploratorio automatizado con ydata-profiling.....	6
2.2. Limpieza de datos.....	7
2.2.1. Eliminación de registros sin CustomerID.....	7
2.2.2. Exclusión de devoluciones ($Quantity \leq 0$).....	7
2.2.3. Exclusión de precios nulos o negativos ($UnitPrice \leq 0$).....	8
2.2.4. Exclusión de cancelaciones (InvoiceNo empezando por "C").....	8
2.2.5. Eliminación de duplicados.....	8
2.3. Verificación post-limpieza.....	9
2.4. Ingeniería de variables (Feature Engineering).....	9
2.4.1. Creación de la variable TotalVenta.....	9
2.4.2. Conversión de la columna de fecha.....	9
2.4.3. Extracción de variables temporales.....	9
3. Regresión: Predicción de Ventas.....	11
3.1. Planteamiento del problema.....	11
3.2. Agregación diaria.....	11
3.3. Ingeniería de variables para series temporales.....	12
3.3.1. Variables de rezago (Lag features).....	12
3.3.2. Media móvil (Rolling mean).....	12
3.3.3. Día de la semana con One-Hot Encoding.....	12
3.4. División cronológica del dataset.....	13
3.5. Selección de características (Feature selection).....	13
3.6. Entrenamiento y comparación de modelos.....	15
3.6.1. Random Forest.....	15
3.6.2. XGBoost.....	15
3.6.3. LightGBM.....	16
3.7. Métricas de evaluación.....	16
3.8. Resultados y selección del modelo.....	16
3.9. Predicción final y exportación de resultados.....	17
3.10. Análisis de resultados.....	17
3.11. Desarrollo Power BI.....	18
4. Conclusiones Regresión.....	21
4.1. Logros alcanzados.....	21
4.2. Próximos pasos.....	21

5. Clustering: Segmentación de Clientes.....	22
5.1. Planteamiento del problema.....	22
5.2. Construcción del perfil del cliente.....	22
5.2.1. Pivotaje del dataset.....	22
5.2.2. Variables construidas.....	22
5.3. Detección y eliminación de outliers con Isolation Forest.....	23
5.4. Escalado de variables con RobustScaler.....	23
5.5. Reducción de dimensionalidad con PCA.....	24
5.6. Elección del número de clusters: método del codo.....	24
5.7. Entrenamiento de los modelos de clustering.....	25
5.7.1. K-Means.....	25
5.7.2. Clustering Jerárquico Aglomerativo.....	25
5.7.3. DBSCAN.....	26
5.8. Evaluación: Silhouette Score.....	26
5.9. Interpretación y perfilado de los segmentos.....	26
5.10. Exportación de resultados para Power BI.....	27
6. Conclusiones Clustering.....	30
6.1. Logros alcanzados.....	30
6.2. Próximos pasos.....	30
Referencias y Recursos.....	31

1. Introducción y Contexto del Proyecto

1.1. Objetivos

Este trabajo forma parte del Módulo de Proyecto (MP6) del Máster en Inteligencia Artificial y Big Data. El objetivo es simular un escenario real de trabajo de un Data Scientist dentro de una empresa de retail, tratando problemas frecuentes en este sector:

- Preprocesado de datos: ajustar el dataset para que los modelos de Machine learning puedan hacer regresiones correctamente.
- Predicción de ventas (regresión supervisada): estimar la demanda futura para optimizar inventarios, campañas de marketing y flujo de caja.

1.2. Repositorio de Github

El repositorio donde están los Jupyter Notebooks con el código empleado y las explicaciones de código más extensas está en este [enlace](#).

1.3. Dataset utilizado

El dataset empleado es el "[E-Commerce Data](#)" disponible en Kaggle y publicado por la Universidad de California. Contiene más de 500.000 transacciones online realizadas entre diciembre de 2010 y diciembre de 2011 por una tienda del Reino Unido especializada en regalos únicos.

Cada fila representa una línea de transacción individual con los siguientes campos:

- **InvoiceNo:** Identificador de factura. Las que empiezan por "C" son cancelaciones.
- **StockCode:** Código de producto.
- **Description:** Nombre del producto.
- **Quantity:** Unidades vendidas. Valores negativos indican devoluciones.
- **InvoiceDate:** Fecha y hora exacta de la transacción.
- **UnitPrice:** Precio unitario en libras esterlinas (£).
- **CustomerID:** Identificador del cliente. Puede estar ausente.
- **Country:** País de origen del cliente.

Los datos están en crudo (raw), lo que significa que contienen inconsistencias, valores nulos y registros anómalos que deben ser tratados antes de cualquier modelado.

1.4. Stack tecnológico

- **Lenguaje:** Python 3.12
- **Entorno:** Jupyter Notebook / VS Code con entorno virtual (.venv)
- **Librerías de análisis:** pandas, numpy
- **Análisis exploratorio automatizado:** ydata-profiling
- **Visualización:** Power BI, matplotlib
- **Machine Learning:** scikit-learn, xgboost, lightgbm

2. Preprocesado de Datos

El preprocesado es la fase más crítica de cualquier proyecto de Data Science. Se calcula que los profesionales del dato dedican alrededor del 60% de su tiempo a esta etapa. La razón es simple: un modelo entrenado con datos sucios aprende patrones erróneos, y ninguna arquitectura de ML, por sofisticada que sea, puede compensar la mala calidad de sus datos de entrada. El principio es "garbage in, garbage out".

2.1. Carga inicial y análisis exploratorio (EDA)

2.1.1. Carga del dataset

El primer paso es cargar el fichero CSV usando la librería **pandas**. Se especifica la codificación `latin1` porque el fichero contiene caracteres especiales (tildes, nombres de productos en francés, etc.) que no son compatibles con UTF-8 por defecto:

```
df = pd.read_csv("data.csv", encoding="latin1")
```

La codificación es un detalle técnico frecuentemente ignorado que puede provocar errores en la carga o que ciertas columnas de texto queden corruptas. Especificarla explícitamente garantiza la integridad de los datos desde el primer momento.

2.1.2. Análisis exploratorio automatizado con ydata-profiling

Antes de realizar ninguna limpieza, se generó un informe automático completo del dataset original mediante la librería **ydata-profiling**:

```
profile = ProfileReport(df, explorative=True)
```

```
from ydata_profiling import ProfileReport
profile = ProfileReport(df, explorative=True)
profile
```

--- Summarize dataset: 100% [00:12:00.00, 4.096s, Completed]

0% [00:00:00.00, 0.00s/1s]
12% [00:00:00.10, 1.52s/1s]
38% [00:00:00.04, 1.12s/1s]
100% [00:00:00.00, 1.00s/1s]

Generate report structure: 100% [00:03:00.00, 3.42s]

Render HTML: 100% [00:00:00.00, 4.64s]

YData Profiling Report

Overview

Overview			
Overview		Alerts	Reproduction
Dataset statistics			
Number of variables	8	Variable types	
Number of observations	541909	Text	3
Missing cells	136534	Numeric	3
Missing cells (%)	3.1%	DateTime	1
Duplicate rows	4879	Categorical	1
Duplicate rows (%)	0.9%		
Total size in memory	173.1 MiB		
Average record size in memory	335.0 B		

Este informe ofrece, para cada columna, estadísticas descriptivas (media, mediana, desviación típica, percentiles), distribución de valores, porcentaje de nulos y alertas automáticas sobre posibles problemas de calidad. Se trata de una herramienta de diagnóstico que permite obtener una visión holística del dataset en segundos, sin necesidad de escribir decenas de líneas de código de exploración manual.

El análisis exploratorio es fundamental porque permite tomar decisiones de limpieza informadas. Sin entender los datos, las decisiones de qué eliminar y qué conservar serían arbitrarias. Gracias al EDA, se detectaron los siguientes problemas principales:

- Presencia de valores nulos en las columnas CustomerID y Description.
- Existencia de transacciones con Quantity negativa (devoluciones).
- Presencia de UnitPrice igual a cero (posibles muestras o errores de registro).
- Facturas que empiezan por la letra "C", indicando cancelaciones de pedido.
- Filas duplicadas que podrían inflar artificialmente las métricas del modelo.

2.2. Limpieza de datos

Una vez diagnosticados los problemas, se procedió a la limpieza de forma sistemática y justificada. Cada decisión de eliminación implica una pérdida de información, por lo que cada acción debe estar argumentada:

2.2.1. Eliminación de registros sin CustomerID

```
df_clean = df_clean.dropna(subset=["CustomerID"])
```

Se eliminan las filas sin identificador de cliente porque, en las fases posteriores de regresión y clustering, la información del cliente es necesaria para construir características de comportamiento. Un registro sin CustomerID no puede ser asociado a ningún patrón de compra individual, por lo que su valor analítico es nulo.

2.2.2. Exclusión de devoluciones (Quantity ≤ 0)

```
df_clean = df_clean[df_clean["Quantity"] > 0]
```

Las cantidades negativas representan devoluciones de productos. Para el modelo de predicción de ventas, interesa conocer cuánto se vende, no cuánto se devuelve. Incluir devoluciones restaría importe a las ventas diarias y distorsionaría la señal que el modelo debe aprender. En un sistema más avanzado, las devoluciones podrían modelarse de forma separada.

2.2.3. Exclusión de precios nulos o negativos (UnitPrice ≤ 0)

```
df_clean = df_clean[df_clean["UnitPrice"] > 0]
```

Un precio igual a cero puede corresponder a muestras gratuitas, errores de entrada de datos o transacciones internas que no representan ventas reales al público. Dado que la variable objetivo que se construirá es el total de ventas en dinero (Quantity × UnitPrice), incluir precios nulos generaría filas con importe cero que contaminarían la distribución de la variable objetivo.

2.2.4. Exclusión de cancelaciones (InvoiceNo empezando por "C")

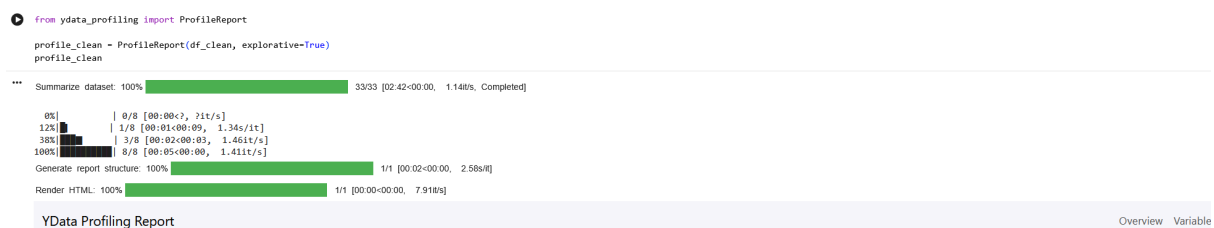
```
df_clean = df_clean[~df_clean["InvoiceNo"].str.startswith("C")]
```

El dataset incluye una convención de nomenclatura: los números de factura que comienzan por la letra "C" corresponden a cancelaciones. Aunque las devoluciones ya se filtran por Quantity < 0, no todas las cancelaciones tienen cantidades negativas en este dataset. Este filtro actúa como una capa adicional de seguridad para asegurarse de que solo se trabaja con transacciones de venta efectiva.

2.2.5. Eliminación de duplicados

```
df_clean = df_clean.drop_duplicates()
```

Los registros duplicados pueden surgir por errores en los sistemas de captura de datos o por fallos en los procesos ETL. Su presencia inflaría artificialmente las métricas de venta y haría que el modelo aprendiese que ciertos patrones ocurren con más frecuencia de la real. La eliminación de duplicados es siempre un paso obligatorio en cualquier pipeline de preprocesado.



Overview

Overview

Alerts 5

Reproduction

Dataset statistics

Number of variables	8
Number of observations	392692
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	128.4 MiB
Average record size in memory	342.9 B

Variable types

Numeric	4
Text	2
DateTime	1
Categorical	1

2.3. Verificación post-limpieza

Tras aplicar todos los filtros, se generó un segundo informe con ydata-profiling sobre el dataset ya limpio para verificar que los problemas detectados inicialmente habían sido resueltos. Esta práctica de doble validación (antes y después) es un hábito profesional recomendable para confirmar que las acciones tomadas han tenido el efecto esperado y no han introducido nuevas anomalías.

2.4. Ingeniería de variables (Feature Engineering)

Una vez limpiados los datos, se procedió a crear nuevas columnas que enriquecen el dataset y aportan información relevante para los modelos.

2.4.1. Creación de la variable TotalVenta

```
df_clean["TotalVenta"] = df_clean["Quantity"] * df_clean["UnitPrice"]
```

Esta es la variable más importante del proyecto: el importe económico de cada línea de transacción. No existe como tal en el dataset original; debe construirse como el producto de la cantidad vendida por el precio unitario. Sin esta variable no es posible calcular las ventas totales por día, que es la variable objetivo del modelo de regresión.

2.4.2. Conversión de la columna de fecha

```
df_clean["InvoiceDate"] = pd.to_datetime(df_clean["InvoiceDate"])
```

La columna de fecha viene almacenada como texto (string). Para poder extraer componentes temporales de ella (día de la semana, mes, semana del año, etc.) es necesario convertirla al tipo datetime de pandas. Sin esta conversión, las operaciones de extracción temporal no son posibles.

2.4.3. Extracción de variables temporales

```
df_clean["Fecha"] = df_clean["InvoiceDate"].dt.date  
df_clean["Mes"] = df_clean["InvoiceDate"].dt.month  
df_clean["DiaSemana"] = df_clean["InvoiceDate"].dt.day_name()
```

Se extrajeron tres variables temporales derivadas de la fecha:

- **Fecha:** La fecha sin componente horaria. Necesaria para agregar transacciones por día.
- **Mes:** El mes del año en formato numérico. Permite capturar la estacionalidad mensual (por ejemplo, picos de ventas en noviembre-diciembre por las fiestas navideñas).
- **DiaSemana:** El nombre del día de la semana. Permite identificar patrones semanales (muchos comercios ven diferencias significativas entre días laborables y fines de semana).

Estas variables temporales son especialmente valiosas en datasets de retail, donde la estacionalidad es un factor determinante del comportamiento de ventas. Al extraerlas explícitamente, le damos al modelo información estructurada sobre el contexto temporal de cada observación.

El resultado final de esta fase es el fichero **dataset_limpio.csv**, que contiene el dataset preprocesado y listo para ser utilizado en las fases de modelado.

3. Regresión: Predicción de Ventas

Una vez obtenido el dataset limpio, se aborda el problema de regresión: predecir las ventas diarias. La predicción de ventas es una de las métricas de mayor valor económico en el sector retail, ya que permite tomar decisiones anticipadas sobre gestión de stock, logística y acciones de marketing.

3.1. Planteamiento del problema

El dataset cubre desde el 1 de diciembre de 2010 hasta el 9 de diciembre de 2011. El objetivo es predecir las ventas diarias del último mes disponible:

- **Conjunto de entrenamiento:** Transacciones del 1 de diciembre de 2010 al 30 de septiembre de 2011.
- **Conjunto de validación:** Transacciones de octubre de 2011. Se usa para comparar modelos y ajustar hiperparámetros sin contaminar el test.
- **Conjunto de test:** Transacciones de noviembre de 2011 en adelante. Simula las predicciones en un escenario real donde el futuro es desconocido.
- **Variable objetivo:** TotalVenta diario (suma de todos los importes de ese día).

La razón de esta partición temporal, en lugar de una partición aleatoria es porque en series temporales, utilizar datos futuros para entrenar el modelo (look-ahead bias) invalida el experimento. El modelo aprendería el futuro, lo cual es imposible en un escenario real. La partición debe respetar siempre el orden cronológico.

3.2. Agregación diaria

```
df_diario = df.groupby('Fecha')[['TotalVenta']].sum().reset_index().sort_values('Fecha')
```

El dataset original tiene registro por transacción individual. El modelo de regresión necesita predecir el total de ventas de cada día como una única cifra. Por eso se agregan todas las transacciones del mismo día sumando sus importes.

Esta transformación es una decisión de diseño clave: se pasa de trabajar con cientos de miles de filas individuales a trabajar con una serie temporal de ~365 puntos (uno por día). Esto reduce el volumen de datos pero conserva la señal que interesa.

3.3. Ingeniería de variables para series temporales

Para que un algoritmo de Machine Learning pueda aprender patrones temporales, no basta con darle la fecha y el valor de ventas. Los algoritmos de ML "no entienden" el tiempo de forma nativa: necesitan que el contexto temporal se codifique como variables numéricas. Por eso se crean las siguientes características:

3.3.1. Variables de rezago (Lag features)

```
df_diario['lag_1'] = df_diario['TotalVenta'].shift(1)
df_diario['lag_7'] = df_diario['TotalVenta'].shift(7)
```

Una variable de rezago (lag) es el valor de la variable objetivo en un momento anterior:

- **lag_1:** Ventas del día anterior. Permite al modelo capturar la inercia inmediata del mercado. Si ayer se vendió mucho, es probable que hoy también.
- **lag_7:** Ventas de hace exactamente 7 días (mismo día de la semana anterior). Es la variable más potente para capturar la estacionalidad semanal, que es muy pronunciada en el comercio minorista.

Sin variables de lag, el modelo carecería de cualquier tipo de "memoria" sobre el pasado y no podría aprender la dinámica temporal de las ventas.

3.3.2. Media móvil (Rolling mean)

```
df_diario['media_7'] = df_diario['TotalVenta'].shift(1).rolling(window=7).mean()
```

La media móvil de los últimos 7 días suaviza el ruido puntual y captura la tendencia reciente. Si una semana entera ha tenido ventas elevadas, la media móvil lo reflejará. Es complementaria a los lags individuales: mientras lag_1 refleja lo que pasó ayer, la media_7 refleja el comportamiento de la semana reciente como conjunto. Se aplica el `shift(1)` previo para evitar usar el valor del día actual en su propio cálculo (filtración de datos futuros).

3.3.3. Día de la semana con One-Hot Encoding

```
df_diario['dia_semana'] = df_diario['Fecha'].dt.dayofweek
df_diario = pd.get_dummies(df_diario, columns=['dia_semana'], drop_first=True)
```

El día de la semana es una variable categórica (lunes, martes, ..., domingo). Los modelos de ML trabajan con números, no con texto. El One-Hot Encoding convierte esta variable en una serie de columnas binarias (0-False o 1-True), una por cada categoría. El parámetro `drop_first=True` elimina una de las columnas (si todos los demás días son 0, se sabe que es el primero).

Este encoding es preferible al Label Encoding (asignar números del 1 al 7) porque evita que el modelo interprete erróneamente que "lunes (1) es la mitad de miércoles (3)" cuando en realidad son categorías sin relación de orden.

3.4. División cronológica del dataset

```
train = df_diario[df_diario['Fecha'] < '2011-10-01'].copy()
val   = df_diario[(df_diario['Fecha'] >= '2011-10-01') & (df_diario['Fecha'] < '2011-11-01')].copy()
test  = df_diario[df_diario['Fecha'] >= '2011-11-01'].copy()
```

Se añade `.copy()` en cada subconjunto para crear copias independientes del DataFrame original y evitar el warning de **SettingWithCopyWarning** de pandas, que puede aparecer si se modifican subsets de un DataFrame sin copiarlos primero.

La elección de octubre como mes de validación es porque tiene la misma duración aproximada que el período de test (noviembre), lo que hace que la estimación del error sobre validación sea representativa del error real esperado sobre test.

3.5. Selección de características (Feature selection)

Las variables finales que se entregan al modelo son:

- `lag_1`: Ventas del día anterior.
- `lag_7`: Ventas de hace una semana.
- `media_7`: Media de ventas de los últimos 7 días.
- `dia_semana_1` a `dia_semana_6`: Variables binarias del día de la semana (no hay `dia_semana_5` porque no hay ventas los Sábados en todo el dataset).

	Fecha	TotalVenta	lag_1	lag_7	media_7	dia_semana_1 \
7	2010-12-09	38193.91	39094.20	46192.49	38923.208571	False
8	2010-12-10	33488.38	38193.91	47197.57	37780.554286	False
9	2010-12-12	17102.35	33488.38	23876.63	35822.098571	False
10	2010-12-13	27578.34	17102.35	31361.28	34854.344286	False
11	2010-12-14	29235.13	27578.34	31009.33	34313.924286	True
..	...	...	...	...	...	...
300	2011-12-05	58081.09	20232.00	16969.25	38257.470000	False
301	2011-12-06	45989.66	58081.09	51632.21	44130.590000	True
302	2011-12-07	69230.60	45989.66	48640.57	43324.511429	False
303	2011-12-08	50395.96	69230.60	41396.79	46265.944286	False
304	2011-12-09	184329.66	50395.96	44405.37	47551.540000	False
	dia_semana_2	dia_semana_3	dia_semana_4	dia_semana_6		
7	False	True	False	False		
8	False	False	True	False		
9	False	False	False	True		
10	False	False	False	False		
11	False	False	False	False		
..	...	...	...	...		
300	False	False	False	False		
301	False	False	False	False		
302	True	False	False	False		
303	False	True	False	False		
304	False	False	True	False		

[298 rows x 10 columns]

Estas variables forman un conjunto bastante informativo, capturan datos a corto plazo, situación semanal y las tendencias. No se incluye la fecha en bruto como variable numérica porque su escala podría distorsionar los modelos de árbol, aunque en la práctica estos son robustos a ello.

3.6. Entrenamiento y comparación de modelos

Se evaluaron tres algoritmos de Machine Learning basados en árboles, que son el estándar para problemas de regresión con datos tabulares. Estos modelos fueron seleccionados por su robustez en el manejo de características no lineales y su buen rendimiento en tareas de predicción de series temporales sin necesidad de una normalización extrema de las variables.

3.6.1. Random Forest

El algoritmo Random Forest es una técnica de *bagging* que consiste en entrenar un gran número de árboles de decisión de forma independiente sobre submuestras aleatorias del conjunto de entrenamiento. Para una predicción, el resultado final se obtiene promediando las salidas de todos los árboles individuales. Este proceso de ensamblaje (o *ensemble*) reduce drásticamente el riesgo de sobreajuste (*overfitting*) que presentan los árboles de decisión únicos, ya que la independencia entre los árboles minimiza la varianza total del modelo. Además de su excelente capacidad predictiva, es conocido por ser el modelo de ensemble más interpretable y sirve como punto de referencia sólido antes de probar modelos de boosting más complejos.

Configuración utilizada: **n_estimators=100, random_state=42**

- **n_estimators=100:** Se seleccionó un total de 100 árboles de decisión. Este número se considera un buen equilibrio: pocos árboles (por ejemplo, 10) harían que el modelo fuera inestable; un número mucho mayor aumentaría el tiempo de entrenamiento sin aportar una mejora significativa en la precisión del modelo.
- **random_state=42:** Se fijó la semilla aleatoria para garantizar que el muestreo de datos y la construcción de los árboles fueran idénticos en cada ejecución, asegurando así la reproducibilidad exacta de los resultados.

3.6.2. XGBoost

XGBoost (eXtreme Gradient Boosting) es un algoritmo de *boosting* que construye árboles de forma secuencial y aditiva. A diferencia de Random Forest, cada nuevo árbol se entrena para corregir los errores residuales (los residuos) cometidos por el conjunto de árboles anterior. Esta corrección iterativa mediante un gradiente descendente permite a XGBoost optimizar el error de forma muy eficiente, lo que se traduce en un rendimiento superior en muchos desafíos de Data Science con datasets de tamaño mediano. No obstante, esta eficiencia conlleva un riesgo de sobreajuste, en hiperparámetros cruciales, como la tasa de aprendizaje, no se ajustan correctamente.

Configuración: **n_estimators=100, learning_rate=0.05, random_state=42**

- **n_estimators=100:** Similar a Random Forest, se definieron 100 árboles a construir de forma secuencial.
- **learning_rate=0.05:** Este hiperparámetro (tasa de aprendizaje) controla la magnitud de la corrección que cada árbol aporta a la predicción final. Un valor bajo (0.05) hace que el proceso de aprendizaje sea más conservador y gradual, lo que ayuda a prevenir el *overfitting* al requerir más árboles para alcanzar el mínimo del error, pero resultando en un modelo más robusto.

3.6.3. LightGBM

LightGBM (Light Gradient Boosting Machine), desarrollado por Microsoft, es una variante avanzada de Gradient Boosting optimizada específicamente para la velocidad y el manejo de grandes volúmenes de datos. Su principal diferencia arquitectónica es que construye los árboles utilizando un enfoque *leaf-wise* (hoja a hoja) con límite de profundidad, en contraposición al enfoque *level-wise* (nivel a nivel) de XGBoost. Esto permite que LightGBM crezca ramas que reducen la pérdida de manera más significativa, encontrando patrones más complejos con menos iteraciones y, por lo tanto, siendo más rápido y, a menudo, más preciso en la práctica.

Configuración: **n_estimators=100, learning_rate=0.05, random_state=42, verbose=-1**

- **verbose=-1:** Se utiliza para suprimir los mensajes de registro internos durante el entrenamiento, garantizando una salida de código limpia y centrada en los resultados.
- **n_estimators=100 y learning_rate=0.05:** Se mantuvieron los mismos parámetros de número de estimadores y tasa de aprendizaje que en XGBoost para una comparación inicial justa y controlada entre los algoritmos de boosting.

3.7. Métricas de evaluación

Para comparar los modelos se utilizan dos métricas complementarias:

- **MAE (Mean Absolute Error):** Error medio absoluto en libras esterlinas. Es la métrica más interpretable: si el MAE es 11.000, el modelo se equivoca de media en £11.000 por día. No penaliza especialmente los errores grandes.
- **RMSE (Root Mean Squared Error):** Raíz del error cuadrático medio. Al elevar los errores al cuadrado antes de promediarlos, penaliza con más fuerza los errores grandes. Un RMSE significativamente mayor que el MAE indica la presencia de días con errores de predicción muy elevados.

Ambas métricas se calculan sobre el conjunto de validación (no sobre test), para evitar seleccionar el modelo basándose en datos que serán usados en la evaluación final.

3.8. Resultados y selección del modelo

Los resultados de la comparativa sobre el conjunto de validación fueron los siguientes:

Resultados en Validación:			
	Modelo	MAE	RMSE
0	RandomForest	12016.348774	15474.027123
1	XGBoost	13875.222907	18734.099031
2	LightGBM	11079.187378	15397.395898

El modelo **LightGBM** obtuvo el mejor rendimiento, con un MAE de £11.079 y un RMSE de £15.397, siendo el algoritmo con menor margen de error tanto promedio como en predicciones extremas.

El criterio de selección es siempre cuantitativo: se elige el modelo con menor error sobre el conjunto de validación, nunca sobre el conjunto de test, para evitar el sesgo de selección.

3.9. Predicción final y exportación de resultados

```
for name, model in models.items():
    test[f'Prediccion_{name}'] = model.predict(X_test)

    output = test[['Fecha', 'TotalVenta', f'Prediccion_{name}']]
    output.to_csv(f'ventas_{name}.csv', index=False)
```

Con los modelos entrenados sobre los datos de train, se generan predicciones sobre el conjunto de test (noviembre 2011). Los resultados se exportan a un fichero CSV con tres columnas: fecha, ventas reales y ventas predichas. Este formato permite:

- Visualizar fácilmente la diferencia entre predicciones y valores reales.
- Calcular métricas finales de evaluación sobre datos que el modelo nunca vio durante el entrenamiento.
- Importar los resultados en herramientas de visualización como Power BI para presentarlos en un Dashboard.

3.10. Análisis de resultados

El fichero **ventas_real_vs_predichas.csv** generado muestra las predicciones frente a los valores reales para el período de noviembre de 2011. Se saca en conclusión:

- El modelo seleccionado, LightGBM, demostró una capacidad robusta para capturar la tendencia general de las ventas diarias y responder de manera efectiva a la estacionalidad semanal. Esta precisión se debe directamente a la inclusión de variables de rezago (**lag_1** y **lag_7**) y la codificación del día de la semana, que proveyeron al algoritmo de la memoria histórica necesaria para entender la dinámica temporal.
- Las métricas finales obtenidas en el conjunto de validación reflejaron este rendimiento: el **Error Absoluto Medio (MAE)** fue de **£11.079**, lo que significa que el modelo se desvía en promedio en esa cantidad por día. El **Error Cuadrático Medio (RMSE)**, sin embargo, fue de **£15.397**, una cifra significativamente mayor.
- La diferencia entre MAE y RMSE indica que, aunque el error promedio es aceptable, existen días con desviaciones muy pronunciadas. Estos errores se concentran en los picos de ventas más extremos, que son difíciles de prever sin información contextual adicional (por ejemplo, promociones flash, eventos, etc.).

- En este sentido, las desviaciones observadas en noviembre de 2011 no son un fallo del modelo, sino una característica intrínseca de la predicción de ventas en retail. Para mejorar la predicción de estos picos atípicos, los próximos pasos deberían enfocarse en el ajuste fino de hiperparámetros y la incorporación de variables exógenas que capturen los eventos extraordinarios.

3.11. Desarrollo Power BI.

Objetivo: A partir del fichero ventas reales_vs_predichas.csv generado en los pasos anteriores, se ha construido un informe en Power BI que visualiza los resultados del modelo. El objetivo es analizar de forma gráfica la calidad del modelo de regresión empleado para predecir ventas diarias, contrastando los valores predichos con los reales y cuantificando el error mediante distintas métricas estadísticas.

Medidas dax utilizadas en la página:

Totales:

- Total ventas reales:

Suma de todas las ventas reales registradas. Convierte el texto a número en la propia agregación.

```
Total Ventas Reales = SUMX( ventas_real_vs_predichas, VALUE(ventas_real_vs_predichas[TotalVenta]) )
```

- Total ventas predichas:

Suma de todas las ventas predichas por el modelo. Misma lógica de conversión.

```
Total Ventas Predichas = SUMX( ventas_real_vs_predichas, VALUE(ventas_real_vs_predichas[Prediccion]) )
```

Métricas de error:

- MAE (Mean Absolute Error)

Promedio del valor absoluto de la diferencia entre venta real y venta predicha. Se interpreta en la misma unidad que las ventas (€). Es la métrica de error más intuitiva.

```
MAE = AVERAGEX( ventas_real_vs_predichas, ABS( VALUE(ventas_real_vs_predichas[TotalVenta]) - VALUE(ventas_real_vs_predichas[Prediccion]) ) )
```

- Mape (Mean absolute Percentage Error)

Error porcentual medio respecto a las ventas reales totales. Permite comparar la precisión del modelo con independencia de la magnitud de las ventas.

```
MAPE = DIVIDE( SUMX(ventas_real_vs_predichas, ABS(VALUE([TotalVenta]) - VALUE([Prediccion]))), SUMX(ventas_real_vs_predichas, VALUE([TotalVenta])) )
```

- RMSE (Root Mean Squared Error)

Raíz del error cuadrático medio. Penaliza más los errores grandes que el MAE, por lo que es útil para detectar si el modelo tiene fallos puntuales muy graves.

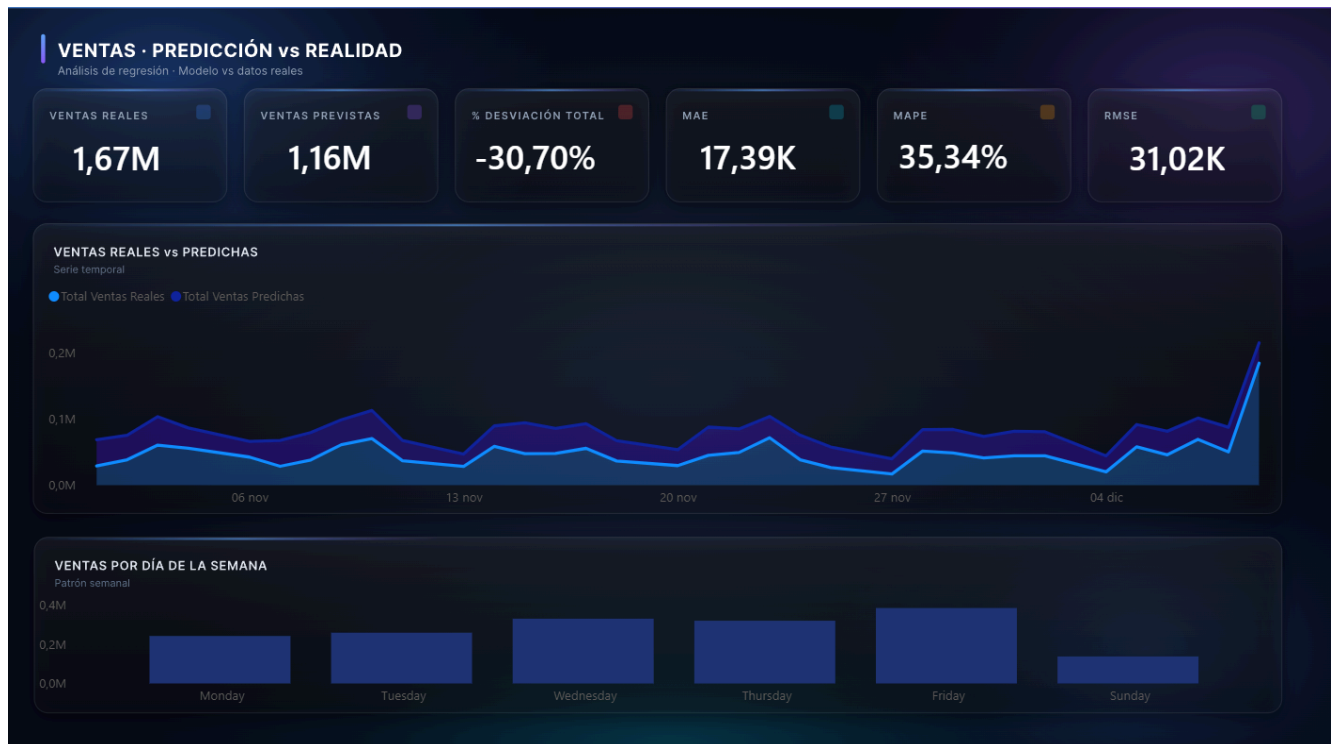
```
RMSE = SQRT( AVERAGEX( ventas_real_vs_predichas, (VALUE([TotalVenta]) - VALUE([Prediccion])) ^ 2 ) )
```

- Desviación total %

Desviación porcentual entre el total predicho y el total real. Muestra de un vistazo cuánto se desvía el modelo en términos acumulados.

% Desviación Total = $\text{DIVIDE}([\text{Total Ventas Predichas}] - [\text{Total Ventas Reales}], [\text{Total Ventas Reales}])$

Visualizaciones:



4. Conclusiones Regresión

4.1. Logros alcanzados

- Se ha consolidado un pipeline de preprocesado robusto que transforma un volumen masivo de datos brutos en una estructura analítica coherente. La exclusión sistemática y justificada de registros anómalos —como cancelaciones, precios de venta nulos y transacciones sin identificador de cliente, ha sido fundamental para garantizar una variable objetivo de ventas libre de ruido. Esta rigurosidad técnica asegura que el modelo aprenda de transacciones comerciales reales, respetando el principio fundamental de "garbage in, garbage out": la precisión de cualquier arquitectura de Machine Learning está limitada por la calidad de sus datos de entrada.
- El Feature Engineering ha desempeñado un papel crítico en el éxito del modelo, destacando especialmente el "lag_7". Esta característica se ha revelado como el factor determinante para capturar la marcada estacionalidad semanal, un patrón dominante en el sector retail. Al proporcionar al algoritmo el valor de ventas de exactamente una semana atrás, el modelo logra internalizar los ciclos naturales del consumo minorista, permitiendo una previsión mucho más ajustada a la realidad del mercado.
- La selección de LightGBM está respaldada por su rendimiento superior en el conjunto de validación, superando a Random Forest y XGBoost tanto en precisión como en eficiencia. La obtención de un Error Absoluto Medio (MAE) de £11,079 se interpreta como un resultado sólido para un modelo baseline inicial. Este nivel de precisión ya proporciona una base sumamente útil para la planificación operativa, permitiendo a la organización tomar decisiones informadas sobre la gestión de inventarios, la logística y el flujo de caja diario.

4.2. Próximos pasos

- Se podrían hacer más pruebas optimizando hiperparámetros: Es necesario realizar un ajuste fino de LightGBM. El objetivo es optimizar parámetros clave como ``learning_rate``, ``max_depth`` y ``num_leaves`` para encontrar el equilibrio óptimo entre sesgo y varianza mejorando la capacidad del modelo ante datos no vistos.
- Exploración de modelos estadísticos complementarios: Aunque se ha optado por modelos basados en árboles por su buen rendimiento, se podría hacer uso de modelos como ARIMA o Prophet como un benchmark adicional.
- Exportar el modelo y hacer uso de él en una aplicación web, para predicciones de precio.

5. Clustering: Segmentación de Clientes

Partiendo del mismo dataset_limpio.csv generado en la entrega anterior, en esta fase se aborda el segundo problema del proyecto: agrupar a los clientes según su comportamiento de compra. A diferencia de la regresión, aquí no hay una variable objetivo que predecir. Se trata de un problema de aprendizaje no supervisado, donde el modelo debe encontrar por sí mismo patrones en los datos y separar a los clientes en grupos con características similares.

El resultado de este bloque tiene un uso de negocio muy directo: una vez identificados los segmentos, el departamento de marketing puede diseñar campañas específicas para cada perfil en lugar de tratar a toda la base de clientes por igual.

5.1. Planteamiento del problema

El dataset limpio tiene granularidad de transacción: cada fila es una línea de compra, y un mismo cliente puede aparecer en cientos de filas distintas a lo largo del año. Para hacer clustering necesitamos justo lo contrario: una sola fila por cliente, con variables que resuman cómo se comporta. Por eso el primer paso es pivotar el dataset, pasando de un fichero de transacciones a un fichero de clientes.

A partir de ahí, se aplica una secuencia de pasos típica en proyectos de clustering: limpiar outliers, escalar variables, reducir dimensiones, decidir el número de grupos y comparar varios algoritmos. El objetivo final es obtener una etiqueta por cliente y un perfil interpretable para cada grupo.

5.2. Construcción del perfil del cliente

5.2.1. Pivotaje del dataset

Se hace un groupby por CustomerID sobre el dataset limpio y se agregan las columnas relevantes para construir las variables de comportamiento:

```
df_clientes = df.groupby("CustomerID").agg({"TotalVenta": "sum", "InvoiceNo": "nunique", "StockCode": "nunique"})
```

Tras renombrar las columnas y añadir el ticket medio, el dataset pasa de 500.000+ filas de transacciones a 4.338 filas de clientes únicos. Esta nueva granularidad es la entrada real del problema de clustering.

5.2.2. Variables construidas

Se han construido cuatro variables que, en conjunto, capturan el comportamiento de compra de cada cliente:

- **TotalGastado:** Suma del importe de todas las compras del cliente. Es el indicador más directo del valor que aporta a la empresa.
- **NumCompras:** Número de facturas distintas (nunique de InvoiceNo). Mide la frecuencia con la que el cliente interactúa con la tienda.

- **ProductosDistintos:** Número de StockCode únicos. Distingue entre clientes que repiten el mismo producto y los que exploran un catálogo amplio.
- **TicketMedio:** Cociente entre TotalGastado y NumCompras. Indica el importe medio que el cliente deja en cada visita.

Estas cuatro variables son suficientes para describir un perfil de comportamiento sin caer en la redundancia. Recoger más columnas (por ejemplo, fechas de compra) habría añadido complejidad sin aportar información nueva, porque la mayor parte de la señal está ya capturada por gasto, frecuencia y variedad.

5.3. Detección y eliminación de outliers con Isolation Forest

En una base de clientes reales siempre aparecen perfiles extremos: mayoristas con pedidos enormes, errores de registro, devoluciones atípicas o cuentas internas. Estos casos sesgan el modelo, porque K-Means tiende a generar clusters pequeños y degenerados solo para acomodar los puntos más alejados del resto.

Para limpiar estos casos se ha utilizado Isolation Forest, un algoritmo de detección de anomalías basado en árboles aleatorios. La idea es sencilla: si un punto se puede aislar del resto en pocas particiones, probablemente es un outlier.

```
iso_forest = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
```

El parámetro `contamination=0.05` indica al algoritmo que asuma que aproximadamente el 5% de los registros son atípicos. Es un valor conservador, dentro del rango habitual del 5-10%, que evita descartar demasiada información y al mismo tiempo elimina los casos más problemáticos.

Tras aplicar el filtro, el dataset queda en 4.121 clientes, eliminando 217 registros considerados extremos. Una vez fuera de los datos, K-Means trabaja sobre la masa principal de la clientela y no tiene que “estirarse” para incluir a un puñado de casos atípicos.

5.4. Escalado de variables con RobustScaler

Las cuatro variables del dataset están en escalas muy distintas: TotalGastado puede llegar a varios miles de libras, mientras que NumCompras suele ser un número de un dígito. Si no se escalan, el algoritmo de clustering interpretaría el gasto como mucho más importante que la frecuencia, simplemente por su rango numérico.

Se ha elegido RobustScaler en lugar del StandardScaler típico. La diferencia está en cómo se centra y se escala cada variable:

- **StandardScaler:** usa la media y la desviación estándar. Ambas son muy sensibles a valores extremos.
- **RobustScaler:** usa la mediana y el rango intercuartílico (IQR), estadísticos que apenas se ven afectados por valores extremos.

Aunque ya se ha aplicado Isolation Forest, la distribución del gasto y el ticket medio sigue teniendo una cola larga (clientes que compran bastante más que la media sin llegar a ser anomalías). Con RobustScaler, esa cola no distorsiona el escalado del

resto. Tras la transformación, cada variable queda centrada en 0 (la mediana) y un valor de 1 representa aproximadamente el percentil 75.

5.5. Reducción de dimensionalidad con PCA

Las cuatro variables construidas tienen una correlación elevada entre sí: los clientes que más gastan suelen ser también los que más compran y los que prueban más productos distintos. En esos casos, parte de la información es redundante.

Se aplica PCA (Análisis de Componentes Principales) sobre el dataset escalado para resumir esas variables en componentes que concentren la varianza:

```
pca = PCA(n_components=0.60, random_state=42)
```

El parámetro `n_components=0.60` le dice a scikit-learn que conserve los componentes necesarios para explicar al menos el 60% de la varianza total. Es un límite razonable: lo suficientemente alto para no perder información útil y lo suficientemente bajo para reducir ruido.

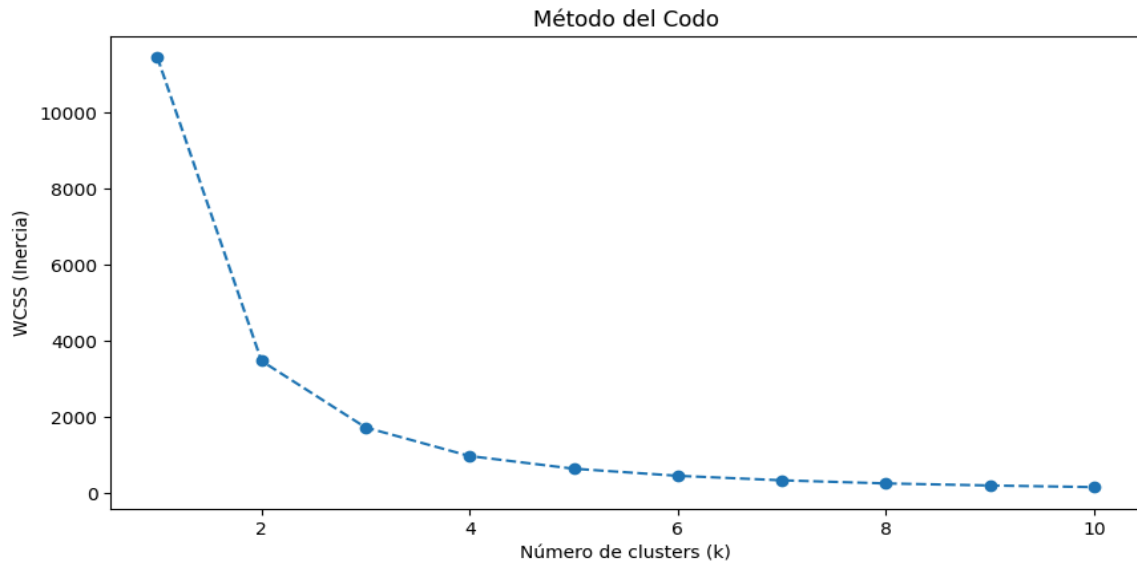
El resultado es revelador: con un único componente principal (PC1) se explica el 62,63% de la varianza. Es decir, la mayor parte de la diferencia entre clientes se puede resumir en un solo eje, que se podría interpretar como un “nivel general de actividad” del cliente: cuanto más arriba en PC1, más gasta, más compra y más productos distintos consume.

Trabajar con PC1 en lugar de las cuatro variables originales tiene tres ventajas prácticas: se elimina el ruido residual, se evita la “maldición de la dimensionalidad” al calcular distancias en K-Means y los clusters se visualizan con mayor claridad.

5.6. Elección del número de clusters: método del codo

K-Means necesita saber por adelantado en cuántos grupos debe partir los datos. Para tomar esa decisión se ha utilizado el método del codo, que consiste en entrenar K-Means con distintos valores de `k` y representar la inercia (suma de distancias al cuadrado dentro de cada cluster) frente al número de clusters.

La inercia siempre baja al añadir más clusters (con `k=Nº` de puntos llegaría a 0), por lo que no sirve fijarse solo en su valor absoluto. El criterio es buscar el punto en el que la curva cambia de pendiente: a partir de ahí, sumar más grupos apenas mejora la compactación.



En la gráfica se observa una caída fuerte hasta $k=3$ y, a partir de ese punto, la mejora marginal se aplana. Por eso se ha elegido $k=3$ como número de clusters. Además del criterio estadístico, tres grupos resultan cómodos de interpretar desde el negocio: clientes de alto valor, clientes recurrentes y clientes ocasionales.

5.7. Entrenamiento de los modelos de clustering

No se ha entrenado un solo algoritmo, sino tres con planteamientos matemáticos distintos. La idea es validar que la estructura encontrada por K-Means no es un artefacto del método, sino una segmentación real de los datos. Si tres enfoques diferentes coinciden en la misma división, los grupos son consistentes.

5.7.1. K-Means

Es el algoritmo central de esta entrega. Funciona por centroides: parte de k puntos iniciales, asigna cada cliente al centroide más cercano y actualiza los centroides como la media de los clientes asignados. Itera hasta que los grupos se estabilizan.

```
kmeans = KMeans(n_clusters=3, init="k-means++", n_init=10, random_state=42)
```

El parámetro `init="k-means++"` mejora la elección de los centroides iniciales para evitar que el algoritmo se quede atrapado en mínimos locales. Con `n_init=10` se repite el entrenamiento diez veces con inicializaciones distintas y se conserva el mejor resultado.

5.7.2. Clustering Jerárquico Aglomerativo

El clustering jerárquico no parte de un número fijo de centroides, sino que construye los grupos “de abajo arriba”: empieza tratando a cada cliente como su propio cluster y, paso a paso, fusiona los más parecidos hasta llegar al número de grupos deseado.

```
hierarchical = AgglomerativeClustering(n_clusters=3)
```


Se ha usado $k=3$ para que el resultado sea directamente comparable con K-Means. La conclusión es que ambos modelos llegan a una segmentación muy parecida, lo cual es una señal de que los grupos existen de manera natural en los datos.

5.7.3. DBSCAN

DBSCAN sigue una lógica distinta: no fuerza un número de clusters, sino que detecta zonas de alta densidad de puntos y deja como “ruido” lo que no encaja. Es útil como contraste: si DBSCAN encontrara muchos clusters pequeños ocultos, sería señal de que K-Means con $k=3$ está simplificando demasiado.

`dbscan = DBSCAN(eps=0.2, min_samples=10)`

En la práctica DBSCAN identifica una masa central muy grande de clientes y unos pocos puntos como ruido en los extremos, sin descubrir subgrupos adicionales relevantes. Ese resultado refuerza la decisión de usar K-Means con $k=3$.

5.8. Evaluación: Silhouette Score

Para comparar los modelos se utiliza el Silhouette Score. Esta métrica mide, para cada punto, lo parecido que es a su propio cluster frente a lo parecido que es al cluster vecino más cercano. Su valor va de -1 a 1:

- Valores cercanos a 1: los clusters están bien separados y son compactos.
- Valores cercanos a 0: los clusters se solapan y la separación es débil.
- Valores negativos: hay puntos asignados al cluster equivocado.

Sobre la columna PC1 obtenida con PCA, los resultados son los siguientes:

- **K-Means:** 0,6184
- **Clustering Jerárquico:** 0,6213

Ambas puntuaciones están por encima de 0,6, lo que indica una segmentación sólida. Como cabía esperar, K-Means y Jerárquico llegan a una calidad prácticamente idéntica, lo que confirma que la estructura de tres grupos está bien presente en el dataset.

El modelo final que se utiliza para etiquetar a los clientes es K-Means, por dos motivos prácticos: produce centroides claros que facilitan la interpretación posterior y es más eficiente cuando se vuelve a entrenar con datos nuevos.

5.9. Interpretación y perfilado de los segmentos

Las etiquetas que devuelve K-Means son simples números (0, 1, 2), así que el siguiente paso es darles un significado de negocio. Para ello se calculan las medias de las variables originales (sin escalar) en cada grupo:

- **Cluster 0 – VIP / Premium (402 clientes):** gasto medio en torno a 4.179 £, casi 10 compras al año y unos 145 productos distintos. Son los clientes más valiosos, los que mueven una parte desproporcionada del volumen total.
- **Cluster 2 – Recurrentes (1.132 clientes):** gasto medio de unas 1.624 £, entre 4 y 5 compras al año y unos 80 productos distintos. Es el grupo

intermedio: clientes ya fidelizados que mantienen una relación estable con la tienda.

- **Cluster 1 – Ocasionales (2.587 clientes):** gasto medio de unas 408 £, entre 1 y 2 compras al año y muy pocos productos distintos. Forman la mayoría de la base de datos y son los clientes con más recorrido potencial.

A partir de estos perfiles aparecen acciones de marketing diferenciadas: programas de fidelización y acceso anticipado para los VIP, recomendaciones personalizadas para los recurrentes y campañas de reactivación para los ocasionales. La segmentación deja de ser un ejercicio técnico y se convierte en una herramienta accionable.

5.10. Exportación de resultados para Power BI

El paso final del notebook consiste en guardar un fichero CSV con toda la información consolidada por cliente. Este fichero es la entrada que utiliza Power BI para construir el dashboard de segmentación.

```
df_final.to_csv("clientes_con_cluster.csv")
```

El CSV resultante contiene, por cada cliente: CustomerID, las cuatro variables de comportamiento sin escalar (TotalGastado, NumCompras, ProductosDistintos, TicketMedio), el identificador numérico del cluster (Cluster_ID) y el nombre comercial del segmento (Segmento). Tener tanto el código como la etiqueta facilita la creación de visualizaciones en Power BI sin necesidad de mapear los grupos a mano cada vez.

Tarjetas KPI

Las seis tarjetas superiores resumen los valores principales del análisis:

- **Total de clientes:** número total de clientes únicos analizados (4.121).
- **Facturación total:** suma de todo lo gastado por estos clientes (4,57M €).
- **Gasto medio:** lo que gasta de media un cliente cualquiera (1.110 €).
- **Ticket medio:** importe medio de una compra individual (333 €).
- **% Clientes VIP:** porcentaje del total que está en el segmento VIP (9,75%).
- **% Facturación VIP:** porcentaje del total facturado que generan los VIP (36,73%).

Gráfico de dispersión · Gasto vs número de compras por cluster

Cada punto representa la posición media de un segmento en dos dimensiones: cuántas compras hace y cuánto gasta. El gráfico demuestra visualmente que los grupos están bien separados. Los VIP aparecen arriba a la derecha (muchas compras, mucho gasto), los Recurrentes en el centro y los Ocasionales abajo a la izquierda. Esta separación confirma que el clustering ha encontrado perfiles realmente distintos.

Donut · Distribución de clientes por segmento

Muestra qué proporción del total representa cada segmento. Los Ocasionales son la gran mayoría con 2.587 clientes (62,78%), seguidos por los Recurrentes con 1.132 (27,47%) y los VIP con 402 (9,75%). Combinado con la KPI de facturación VIP, evidencia que pocos clientes concentran gran parte de los ingresos.

Gráfico de barras · Facturación por segmento

Compara la facturación total que aporta cada grupo. Los Recurrentes lideran ligeramente con 1,84M €, seguidos muy de cerca por los VIP con 1,68M € (aunque estos son cuatro veces menos numerosos) y los Ocasionales con 1,06M €. Es la lectura económica de la segmentación: dónde está realmente el dinero.

Totales

- **Total Clientes**

Cuenta el número total de clientes únicos en el dataset. Es la base sobre la que se calculan todos los porcentajes.

```
Total Clientes = COUNTROWS( clientes_con_cluster )
```

- **Facturación Total**

Suma de todo el gasto registrado por los clientes. Es el volumen total de negocio del período analizado.

```
Facturacion Total = SUM( clientes_con_cluster[TotalGastado] )
```

Promedios por cliente

- **Gasto Medio Cliente**

Lo que gasta de media un cliente cualquiera en todo el período. Útil para comparar el valor medio de cada segmento.

```
Gasto Medio Cliente = AVERAGE( clientes_con_cluster[TotalGastado] )
```

- **Ticket Medio Global**

Importe medio de una compra individual. Indica el valor típico de cada transacción.

```
Ticket Medio Global = AVERAGE( clientes_con_cluster[TicketMedio] )
```

- **Compras Medias**

Número medio de compras realizadas por cliente. Mide la frecuencia de compra del segmento.

Compras Medias = AVERAGE(clientes_con_cluster[NumCompras])

- **Productos Medios**

Número medio de productos distintos comprados por cliente. Mide la variedad del consumo.

Productos Medios = AVERAGE(clientes_con_cluster[ProductosDistintos])



6. Conclusiones Clustering

6.1. Logros alcanzados

- Se ha pivotado el dataset de transacciones a clientes, pasando de más de 500.000 filas a 4.338 perfiles de cliente con cuatro variables de comportamiento.
- Se han eliminado 217 clientes atípicos con Isolation Forest (contaminación del 5%), dejando un dataset robusto de 4.121 clientes.
- Las variables se han escalado con RobustScaler para mitigar el efecto de la cola larga y los datos se han reducido a un único componente principal que explica el 62,63% de la varianza.
- El método del codo indica con claridad $k=3$ como número óptimo de clusters, decisión confirmada después al comparar K-Means con clustering jerárquico y DBSCAN.
- El modelo final obtiene un Silhouette Score de 0,6184 (K-Means) y 0,6213 (Jerárquico), valores que indican una segmentación consistente y bien separada.
- Los tres clusters se han traducido a perfiles de negocio claros (VIP, Recurrentes y Ocasionales) sobre los que el equipo de marketing puede diseñar acciones concretas.

6.2. Próximos pasos

- Se podrían añadir variables RFM (Recency, Frequency, Monetary) más elaboradas, incorporando el tiempo desde la última compra para refinar los perfiles.
- Probar variantes adicionales como K-Means con más componentes principales o Gaussian Mixture Models para validar la robustez de los tres segmentos.
- Cruzar los clusters con la información geográfica (columna Country) para detectar si ciertos perfiles están sobrerrepresentados en mercados concretos.
- Conectar la salida del clustering con el modelo de regresión para estimar la contribución de cada segmento a las ventas diarias previstas.

Referencias y Recursos

- Dataset original: E-Commerce Data — UCI Machine Learning Repository / Kaggle
- Guía del proyecto: MP6 Tema 1 — Introducción al proyecto
- Guía del proyecto: MP6 Tema 2 — Proyecto de Regresión
- Documentación scikit-learn: https://scikit-learn.org/stable/supervised_learning.html
- Documentación LightGBM: <https://lightgbm.readthedocs.io/>
- Documentación XGBoost: <https://xgboost.readthedocs.io/>
- ydata-profiling: <https://docs.profiling.ydata.ai/>
- Repositorio de Github del proyecto: <https://github.com/lucaschacon3/TFM-E-Commerce-Sales-Forecast>
- Documentación Pandas: <https://pandas.pydata.org/docs/>
- Documentación NumPy (cálculo numérico): <https://numpy.org/doc/>
- Visualización: Documentación Microsoft Power BI